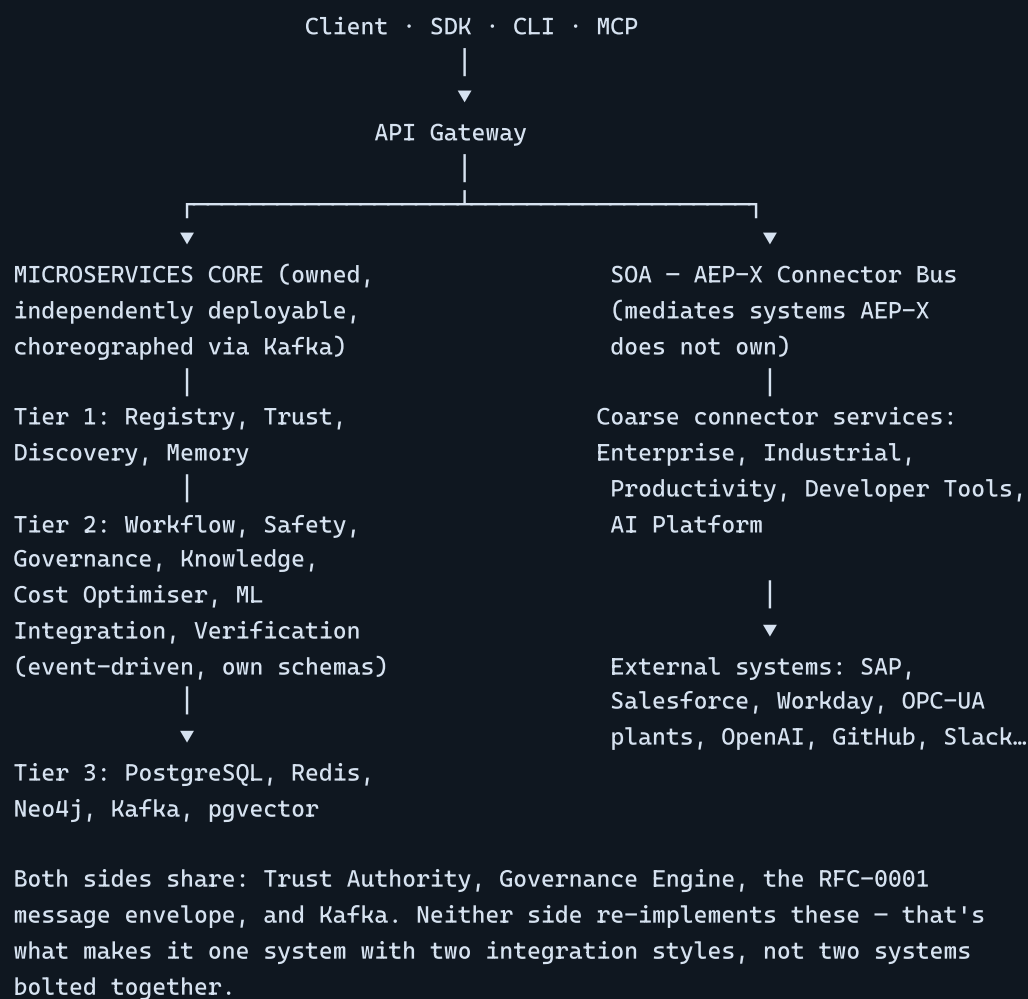


AEP-X — Hybrid Architecture & SOA-vs-Microservices Comparison

One platform, two integration styles, sharing a trust & governance spine

Companion to [SOA-Architecture.html](#), [Microservices-Architecture-v2.html](#) Prepared 7 July 2026

1. The Hybrid, in One Diagram



2. Why the Boundary Sits Exactly at "Who Owns the System"

The deciding question: **does AEP-X control the release cadence and protocol of the thing being integrated?**

- Registry, Trust, Memory, Workflow, Safety, Governance, Knowledge, Cost Optimiser, ML Integration, Verification — **yes**. Independent deployability and choreography pay off because each changes on its own schedule.
- SAP, Salesforce, Workday, an OPC-UA-speaking factory PLC — **no**. What's needed is one place enforcing trust/policy/audit consistently and translating heterogeneous protocols into one canonical shape.

3. Comparison Table

Dimension	Microservices (core)	SOA / ESB (connector layer)
Coupling	Loose — own schema, know only APIs	Loose to externals (adapter hides protocol), tighter internally (bus mediates every call)
Deployment cadence	Independent, as often as needed	Per category, usually far less often
Scaling unit	Per-service	Per-category
Protocol standardisation	One internal standard	Many external protocols in, one canonical envelope out
Governance enforcement	Distributed by convention	Centralised, structurally enforced
Failure blast radius	Contained to one service	Contained to one connector category
Cost to build N integrations	High if used for externals — N near-identical services	Low — N adapters inside 5 coarse services
Fit for AIA-R0–R4 compliance	Adequate for internal services	Better — AIA-R is naturally per-connector
Anti-pattern risk if misapplied	"Distributed monolith" if DBs are shared	"God bus" if the bus grows business logic

4. What NOT to Build

- **No commercial ESB product** (MuleSoft, BizTalk). The Connector Bus is a ~200-line FastAPI service plus the existing Kafka bus.
- **Don't build all 25+ external systems** before validating one per category (Salesforce, GitHub, Slack, OpenAI first).
- **Don't let the SOA layer duplicate governance logic** — it calls existing Trust Authority / Governance Engine APIs, never grows its own.
- **Don't re-open Marketplace Engine or connector deferrals** to justify this work.

5. Recommendation

Finish the Microservices v2 additions first (Knowledge, Verification, Cost Optimiser, ML Integration — these unlock the platform's core economic and anti-hallucination claims), then start the SOA Connector Bus (Weeks 13+, gated on Trust Authority + Governance Engine being healthy in production). The two are not in tension for engineering capacity — the SOA layer's first cut reuses the core's existing trust/policy/audit APIs rather than designing a second platform from scratch.